

向量数据库与 Embedding 技术

Embedding 把 " 语义 " 变成 " 向量 " ， 向量数据库让 " 找相似 " 变成 " 查索引 " ——二者共同构成了 RAG 系统的基石。

向量数据库与 Embedding 技术

Embedding 把 " 语义 " 变成 " 向量 " ， 向量数据库让 " 找相似 " 变成 " 查索引 " ——二者共同构成了 RAG 系统的基石。

- - -

Embedding 的本质与作用

Embedding（嵌入） 是将离散的非结构化数据（文本、图像、音频）映射为连续稠密向量（Dense Vector）的技术。其本质是学习一个映射函数 $f: x \rightarrow \mathbb{R}^d$ ，使得语义相近的输入在向量空间中

直观理解：Embedding 把人类理解的 " 意义 " 翻译成了机器能计算的 " 坐标 "。两段文本语义越接近，其向量夹角越小；语义无关，则

" 猫在睡觉 " $\rightarrow [0.21, -0.55, 0.83, \dots]$ $\cap$
" 小猫入眠 " $\rightarrow [0.19, -0.51, 0.80, \dots]$ $\perp$ 余弦相似度 0.97（语义相近）
" 今天股市大跌 " $\rightarrow [-0.42, 0.11, 0.05, \dots]$ 余弦相似度 0.08（语义无关）

Embedding 在 AI 系统中的作用：

作用 说明 应用场景

语义检索 用向量相似度替代关键词匹配 RAG、搜索引擎

聚类分析 向量空间中相近样本归为一类 用户分群、文档分类

推荐系统 物品与用户向量内积排序 内容推荐

去重判定 向量距离判断内容近似 文档去重、抄袭检测

特征表示 作为下游模型的输入特征 分类、回归任务

```
from openai import OpenAI
client = OpenAI()
```

文本转向量

```
resp = client.embeddings.create(
    model="text-embedding-3-small",
```

```
input = "大模型推理优化的关键是什么?"
)
vector = resp.data[0].embedding # 1536 维向量
print(len(vector), vector[:5])

- - -
```

主流 Embedding 模型 (Word2Vec、BERT、text-embedding-3、BGE)

1. Word2Vec (2013)

最早的词向量模型，通过 CBOW 或 Skip-gram 在大规模语料上训练，得到每个词的静态向量。它开创了 "分布式表示" 范式，但每个词只有一个向量，无法处理上下文。

2. BERT (2018)

基于 Transformer 的上下文相关向量。同一词在不同句子中向量不同，解决了多义词问题。常用 [CLS] 位置或均值池化得到句向量。但 BERT 原生并非为检索优化，语义检索效果一般，需配合对比学习微调 (Sentence-BERT)。

3. text-embedding-3 (OpenAI, 2024)

OpenAI 最新一代商用 Embedding 模型，分为 small (1536 维) 和 large (3072 维)，支持维度裁剪 (Matryoshka Representation)。在 MTEB 榜单上表现优异。

4. BGE (BAAI General Embedding, 2023 - 2024)

智源研究院开源的中英双语 Embedding 模型系列，在 MTEB 中文榜单长期领先。支持 bge-large-zh、bge-m3 (多语言、多功能、多粒度) 等变体，可本地部署，是中文 RAG 的首选。

模型 类型 维度 是否开源 适用场景

Word2Vec	静态词向量	100 - 300	是	传统 NLP、教学
BERT / SBERT	上下文句向量	768	是	通用语义、微调基座
text-embedding-3	句向量 (API)	1536 / 3072	否	快速集成、英文为主
BGE / BGE-M3	句向量 (中英)	768 - 1024	是	中文 RAG、本地部署
E5 / GTE	句向量 (多语言)	768 - 1024	是	多语言检索

选型口诀：英文闭源选 OpenAI，中文开源选 BGE，多语言选 BGE-M3 或 E5，需要精确语义选 SBERT。

- - -

向量相似度计算（余弦、点积、欧氏）

向量检索的核心是比较向量间的 " 距离 " 或 " 相似度 "。

1 . 余弦相似度 (C o s i n e S i m i l a r i t y)

衡量向量方向的一致性，对向量长度不敏感，是语义检索最常用的指标。

$$\cos(A, B) = (A \cdot B) / (|A| \times |B|)$$

2 . 点积 (D o t P r o d u c t)

当向量已归一化（单位向量）时，点积等于余弦相似度，计算更快。许多向量数据库默认使用点积以提升性能。

3 . 欧氏距离 (E u c l i d e a n D i s t a n c e)

衡量向量在空间中的绝对距离，对长度敏感。适用于需要考虑向量模长的场景（如图像特征）。

```
import numpy as np

def cosinesim(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def dotproduct(a, b):
    return np.dot(a, b)

def euclidean(a, b):
    return np.linalg.norm(a - b)

a = np.array([1.0, 2.0, 3.0])
b = np.array([2.0, 4.0, 6.0])
print("余弦:", cosinesim(a, b)) # 1.0 (方向一致)
print("点积:", dotproduct(a, b)) # 28.0
print("欧氏:", euclidean(a, b)) # 3.74
```

指标	是否归一化敏感	适用场景	备注
----	---------	------	----

余弦相似度 否 文本语义检索 最常用

点积 是（归一化后 = 余弦） 性能优先的大规模检索 需先归一化

欧氏距离 是 图像、聚类 对模长敏感

- - -

向量数据库对比（FAISS、Milvus、Pinecone、Chroma、Weaviate、Qdrant）

向量数据库专门用于存储、索引和检索海量高维向量，是 RAG 系统的 " 存储引擎 "。

数据库 类型 部署方式 核心特点 适用场景

FAISS 库（非服务） 本地嵌入 极致性能、纯索引、Meta 开源 原型验证、单机批量检索

Milvus 分布式数据库 自托管 / 云 可水平扩展、亿级向量、生态完善 企业级大规模生产

Pinecone 全托管 SaaS 云 API 零运维、按量付费 快速上线、不想运维

Chroma 轻量数据库 本地 / 嵌入式 Python 友好、开发体验好 中小项目、开发期

Weaviate 混合数据库 自托管 / 云 内置向量化模块、GraphQL 需要混合检索

Qdrant 高性能数据库 自托管 / 云 Rust 编写、过滤性能强 元数据过滤频繁的场景

Chroma 快速上手

```
import chromadb
client = chromadb.Client()
collection = client.create_collection("docs")

collection.add(
    documents=["RAG 是检索增强生成", "Embedding 把文本变成向量"],
    metadatas=[{"src": "doc1"}, {"src": "doc2"}],
    ids=["1", "2"]
)

results = collection.query(querytexts=["什么是 RAG"])
```

选型建议：原型用 Chroma / FAISS，企业生产用 Milvus，免运维用 Pinecone，Qdrant，需混合检索用 Weaviate。

- - -

ANN 索引算法（HNSW、IVF、PQ、LSH）

当向量规模达百万、亿级时，精确检索（暴力计算所有距离）无法满足延迟要求。近似最近邻（ANN，Approximate Nearest Neighbor）算法以少量精度损失换取数量级的速度提升。

1. HNSW (Hierarchical Navigable Small World)

分层的图索引，通过 "小世界图" 实现高效导航。查询时从顶层粗粒度快速逼近，逐层下沉精细搜索。召回率高、查询快，但内存占用大，是当前最主流的默认索引。

2. IVF (Inverted File)

基于聚类的倒排索引。先用 K-means 将向量空间划分为 `nlist` 个簇，查询时只搜索最近的一个簇。通过调节 `nprobe` 可在速度与召回间权衡。

3. PQ (Product Quantization)

将高维向量切分为若干子段，每段独立量化压缩，大幅降低内存占用。常与 IVF 组合为 IVFPQ，适用于内存受限的超大规模场景。

4. LSH (Locality-Sensitive Hashing)

局部敏感哈希，将相近向量以高概率映射到同一桶。理论优雅但在高维下效果一般，实际工程中使用较少。

HNSW：图结构导航 → 高召回、高内存（默认首选）

IVF：聚类倒排 → 可调参、均衡（大规模通用）

PQ：乘积量化压缩 → 省内存、有精度损失（十亿级+内存受限）

LSH：哈希分桶 → 理论好、实际一般（教学/特定场景）

FAISS 中构建 HNSW 索引

```
import faiss
```

```
import numpy as np
```

```
dim = 768
```

```
index = faiss.IndexHNSWFlat(dim, 32) # M=32 每节点连接数
```

```
index.hnsw.efConstruction = 40 # 构建时搜索宽度
```

```
index.hnsw.efSearch = 16 # 查询时搜索宽度
```

```
vectors = np.random.random((100000, dim)).astype('f')
```

```
index.add(vectors)
```

```
D, I = index.search(vectors[:5], k=10) # 查询 Top - 10  
- - -
```

混合检索（向量 + 关键词 + 重排）

纯向量检索擅长语义匹配，但对专有名词、编号、精确词匹配较弱；纯关键词检索（BM25）恰好互补。混合（BM25 + 向量语义检索）融合二者优势，再配合重排序模型，是当前 RAG 的最佳实践。

用户查询

└──→ BM25 关键词检索 → Top - K 候选

└──→ 向量语义检索 → Top - K 候选

↓ 融合（RRF 或加权）

合并候选集

↓ Cross - Encoder 重排

精选 Top - N

↓ 送入 LLM

融合策略

- RRF（Reciprocal Rank Fusion）：按各路检索的排名倒数求和，简单稳健，无超参。
- 加权融合：对归一化后的分数加权求和，需调参。

重排序（Rerank）

检索阶段用 Bi - Encoder（快但粗），重排阶段用 Cross - Encoder（慢但精）。Cross - Encoder 能捕捉 query 和 doc，能捕捉细粒度交互。常用模型：bge-reranker、Cohere Rerank。

```
from FlagEmbedding import FlagReranker  
  
reranker = FlagReranker('BAAI/bge-reranker-large')  
pairs = [['查询', '文档1'], ['查询', '文档2']]  
scores = reranker.compute_score(pairs) # 重排分数  
- - -
```

RAG 中的 Embedding 选型策略

RAG 系统的检索质量，很大程度上由 Embedding 模型决定。选型应综合考虑以下因素：

维度 考量点 建议

语言 中文 / 英文 / 多语言 中文选 BGE，多语言选 BGE - M3 / E5

领域 通用 vs 专业 通用用预训练模型，专业领域需微调

部署 本地 vs API 数据敏感 / 离线选本地开源，快速验证用 API

维度 召回率 vs 性能 高维召回好但慢，可用维度裁剪平衡

长度 文档块长度 长文档选支持长上下文的模型（BGE - M3 支持 8192）

实践要点

1. Query 与 Document 用同一模型：检索双方必须使用同一 Embedding 空间
2. 评测先行：用领域问答集评测 Recall@K，而非盲信榜单。MTEB 第一不等于你的业务第一
3. 切块配合模型：短块（128 - 256 token）适合精确匹配，长块（512+）适合长上下文模型
4. 必要时微调：当通用模型召回不足时，用对比学习在领域数据上微调（如 LLaMA-Factory、sentence-transformers 的 MultipleNeg
5. 建立基线再优化：先用 BM25 建立检索基线，再叠加向量检索与重排，量化每一步的增益。

评测检索质量示例

```
def evaluator_recall(retriever, queries, groundtruth):
    hits = 0
    for q, gt in zip(queries, groundtruth):
        docs = retriever.search(q, topk=k)
        if gt in [d.id for d in docs]:
            hits += 1
    return hits / len(queries)

recall = evaluator_recall(retriever, queries, gt_ids)
print(f"Recall@5 = {recall:.2%}")
```

关键词：Embedding，向量数据库，FAISS，Milvus，HNSW，ANN，混